CO4-AT3- Comparative analysis

Name: Ajay Kumar Reddy

Reg.no:192425274

Course: CSA0712 - COMPUTER NETWORKS

Faculty: Dr. S. SALINI

1. Multiplexing vs. Demultiplexing

At the sending host, the transport layer must combine data streams from many independent application sockets onto the single network-layer channel available to the host; at the receiving host, the reverse problem is to correctly sort incoming segments back out to the right socket. These two complementary operations bookend every transport-layer exchange.

Column 1 — Multiplexing (Sender)	Column 2 — Demultiplexing (Receiver)
Gathering: Each active application (a browser tab, a DNS resolver, an SSH session) owns a socket, identified locally by a port number and, for TCP, by the full 4-tuple of source/destination IP and port. **Labeling:** The transport layer takes data handed down by each socket's send buffer and stamps it with a transport-layer header containing the source port (so replies can be routed back) and the destination port (so the remote host know which service to deliver to). **Segmentation:** Data is broken into segments (TCP) or datagrams (UDP) no larger than the path's usable size, each carrying its own header copy. **Encapsulation:** The completed segment is passed to the network layer, which wraps it in an IP header before transmission — the transport layer itself never touches IP addressing logic directly, only ports. **Result:** One outbound link now safely interleaves traffic from many logically separate sockets without ambiguity.	**Extraction:** The receiving transport layer strips the IP header (handled by the network layer) and reads the transport segment header to recover source and destination port fields. **Identification:** For UDP, the (destination IP, destination port) pair alone identifies the target socket. For TCP, the full 4-tuple (source IP, source port, destination IP, destination port) is required, since a busy server may hold many simultaneous TCP sockets bound to the same local port but different remote peers. **Socket lookup:** The operating system's socket table maps this tuple to a specific socket descriptor and its associated receive buffer. **Delivery:** The payload is copied into that socket's buffer, waking the owning application/process if it is blocked on a read() call. **Error handling:** If no socket matches (e.g., a UDP datagram to a closed port), the host typically returns an ICMP port-unreachable message rather than silently dropping it.

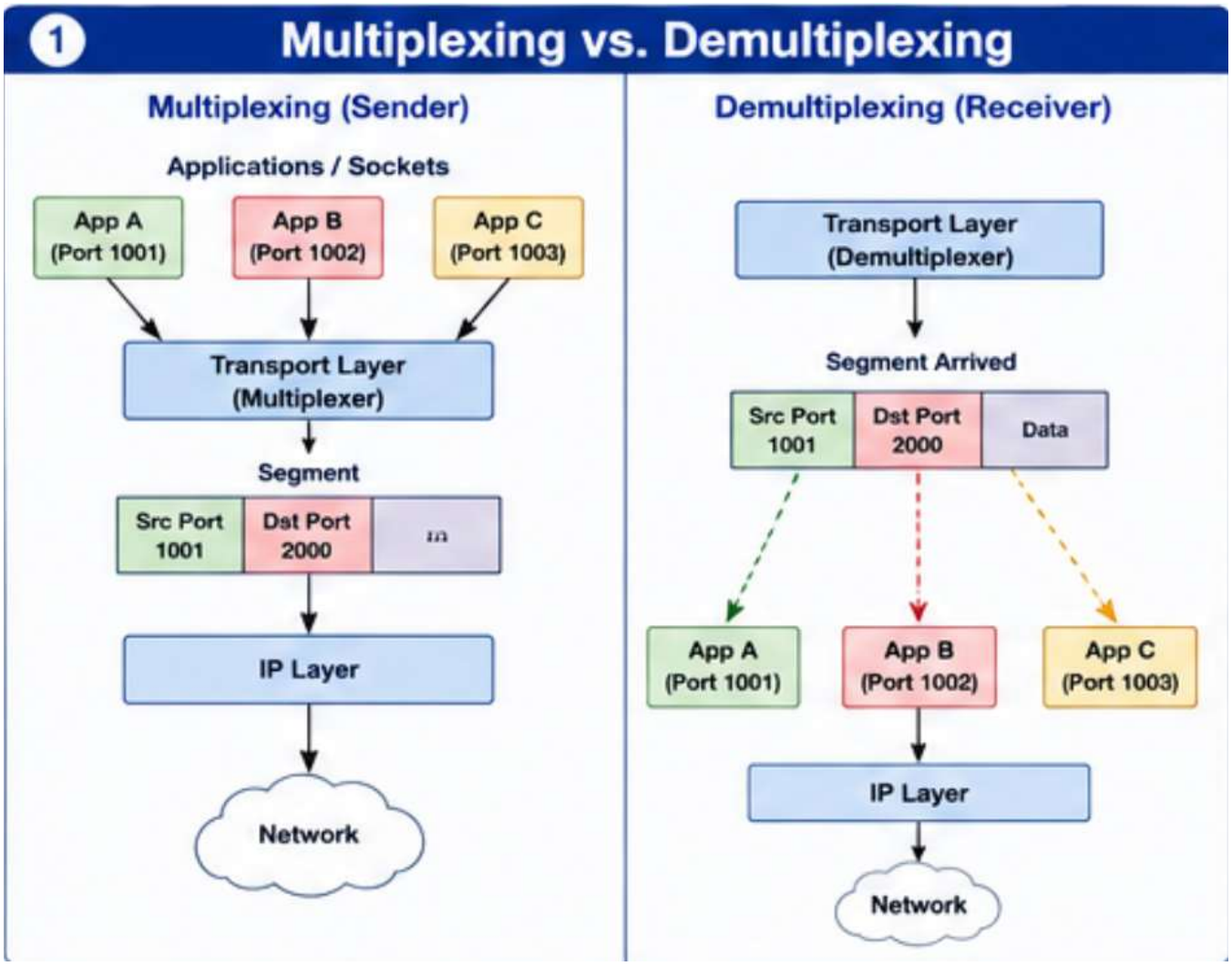


Figure 1. Sender-side multiplexing of three application sockets into labeled segments, and receiver-side demultiplexing back to the matching sockets using port/IP identifiers.

Comparative Synthesis

Multiplexing and demultiplexing are mirror-image operations that exist purely because a single network interface must be shared among many concurrent application-layer conversations. Multiplexing solves the fan-in problem at the source; demultiplexing solves the fan-out problem at the destination. Both rely on the same shared vocabulary — port numbers embedded in the transport header — but demultiplexing carries the harder task, since it must distinguish among many possible sockets using information that arrives already interleaved with other flows, whereas multiplexing simply appends known local information at send time.

Practical Implications

In practice, this pairing is why a single server can run web, mail, and DNS services simultaneously on one network interface, and why port numbers below 1024 are conventionally reserved for well-known services — demultiplexing depends entirely on both ends agreeing, implicitly or explicitly, on which port identifies which service.

2. TCP Sockets vs. UDP Datagrams

TCP and UDP occupy the same layer of the protocol stack but expose fundamentally different socket abstractions to applications: one manages a long-lived, ordered conversation; the other treats every unit of data as an independent, fire-and-forget message.

Column 1 — TCP Socket State	Column 2 — UDP Datagram Socket
Connection-oriented: A TCP socket does not become usable for data transfer until a three-way handshake (SYN, SYN-ACK, ACK) establishes shared initial sequence numbers between both endpoints.	**Connectionless:** A UDP socket can send a datagram to any destination the instant it is bound — there is no handshake and no notion of an established session state.
Stream abstraction: TCP presents data as an unstructured, ordered byte stream — the application's original message boundaries are not preserved on the wire, and the receiver must layer its own framing on top if needed.	**Message-oriented:** Each call to send() produces exactly one datagram, and the receiver's corresponding recv() call returns exactly one message boundary-preserved unit, never a partial or merged message.
Stateful bookkeeping: Each socket maintains sequence and acknowledgment counters, a congestion window, a receive window, retransmission timers, and buffered-but-unacknowledged data — all of which must persist for the lifetime of the connection.	**Stateless operation:** The kernel does not track sequence numbers, acknowledgments, or windows for a UDP socket; each datagram is handled independently of all others.
Reliability guarantees: Lost segments are detected via timeout or duplicate ACKs and retransmitted; out-of-order segments are buffered and resequenced before delivery to the application.	**No delivery guarantees:** Datagrams may be lost, duplicated, or arrive out of order, and UDP itself makes no attempt to detect or correct this — any needed reliability must be built at the application layer.
Graceful teardown: Closing a TCP socket involves an explicit FIN/ACK exchange in each direction, allowing either side to finish sending before the connection fully closes.	**Minimal overhead:** The UDP header is a fixed 8 bytes (versus TCP's minimum 20), and the absence of handshake/teardown steps yields lower latency for short, latency-sensitive exchanges (DNS lookups, VoIP, gaming).

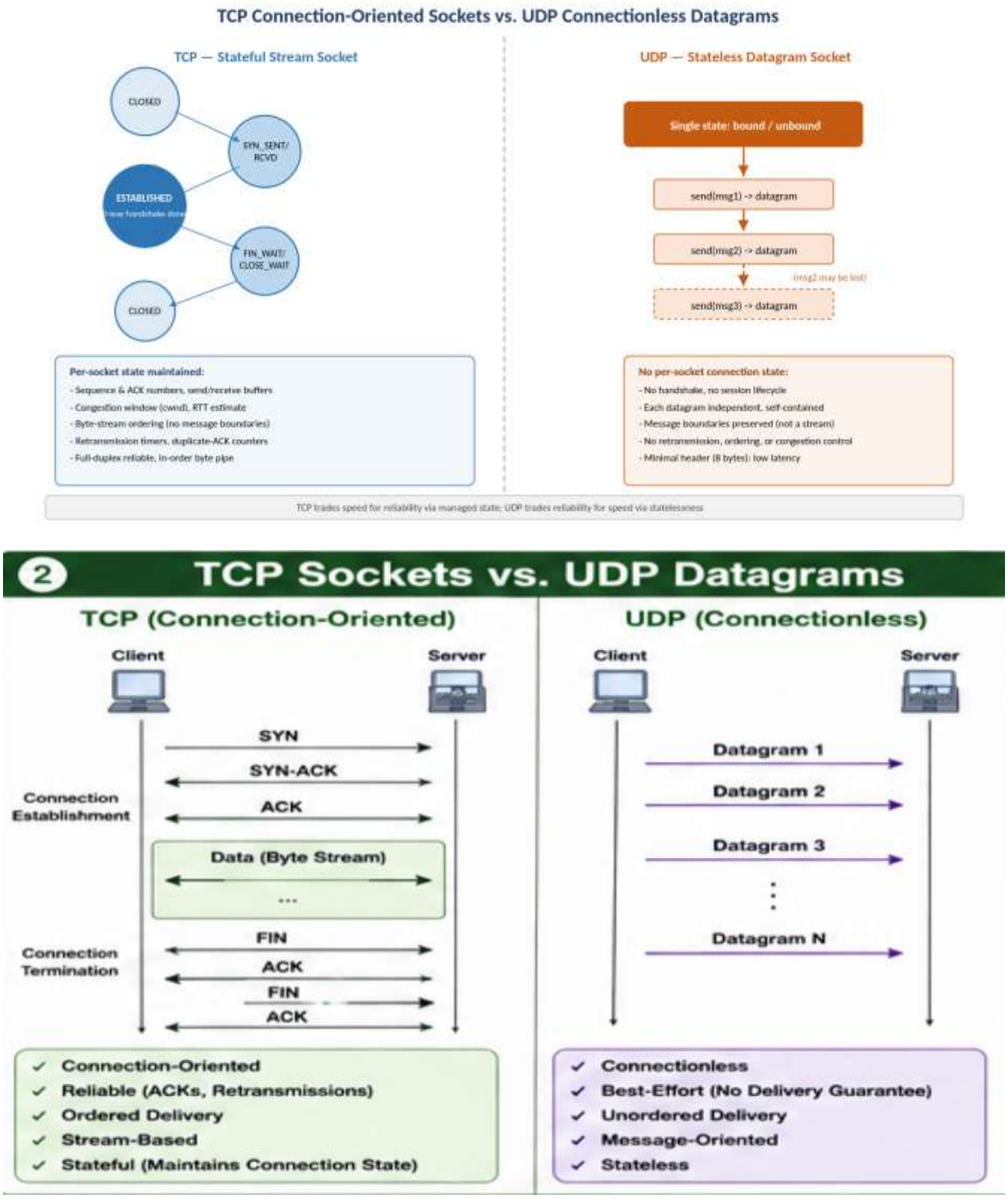


Figure 2. TCP's stateful connection lifecycle (handshake through teardown) contrasted with UDP's stateless, independent per-datagram send path.

Comparative Synthesis

The distinction reduces to a fundamental engineering trade-off between reliability and overhead. TCP invests in per-socket state so that applications can treat the network as a dependable, ordered pipe — an abstraction well suited to file transfer, web pages, and email, where correctness matters more than a few milliseconds of delay. UDP discards that state to minimize latency and per-packet cost, delegating reliability decisions to the application, which is appropriate when timeliness matters more than completeness, or when the application already implements its own reliability semantics (as QUIC does, discussed in a later section).

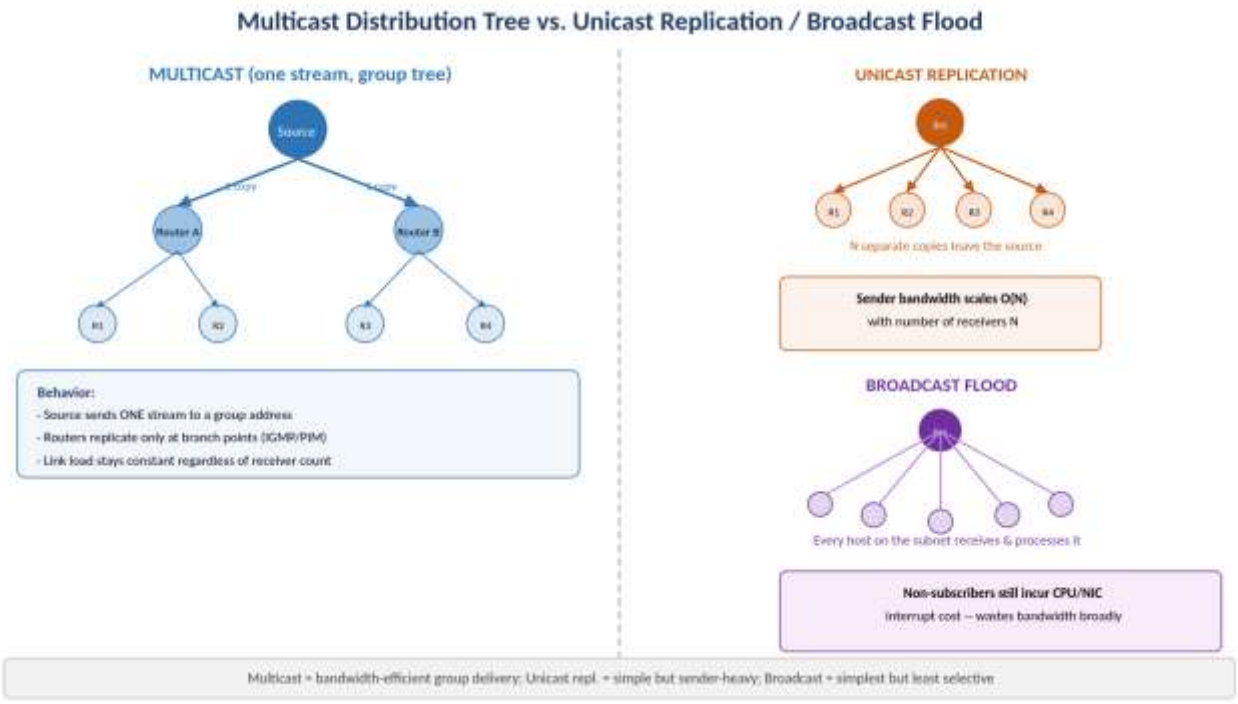
Practical Implications

System designers choose between the two by asking whether a stale retransmission is worse than no data at all: a video call drops a frame and moves on (UDP-friendly), while a bank transfer cannot tolerate a silently dropped byte (TCP-mandatory). Many modern systems, including QUIC itself, now build custom reliability semantics on top of UDP rather than accepting TCP's fixed guarantees wholesale.

3. Multicast vs. Unicast/Broadcast (Transport Context)

When one sender must reach many receivers, the network can choose among three fundamentally different delivery strategies, each with a different bandwidth and scalability profile.

Column 1 — IP Multicast	Column 2 — Unicast Replication & Broadcast
Group addressing: Receivers join a multicast group identified by a class-D IP address (e.g., 224.0.0.0/4) using IGMP (IPv4) or MLD (IPv6); the sender transmits to that group address without needing to know individual receiver addresses. **Single logical stream:** The source emits exactly one copy of each UDP packet; typically layered over UDP because multicast has no meaningful notion of a single bidirectional connection. **Tree-based replication:** Multicast-aware routers (using protocols such as PIM) build a distribution tree and duplicate packets only at branch points where the tree splits toward different subnets — links shared by multiple downstream receivers carry only one copy. **Bandwidth efficiency:** Because replication happens deep in the network rather than at the source, the sender's uplink load is independent of the number of subscribed receivers. **Use cases:** IPTV distribution, financial market data feeds, and routing-protocol updates that must reach many listeners simultaneously.	**Unicast replication:** To reach N receivers without multicast support, the source (or an application-layer relay) must open N separate point-to-point connections and transmit a full copy of the data on each. **Sender-side cost:** Because every copy originates from the same sender, its outbound bandwidth and CPU/encoding cost both scale linearly, O(N), with the number of receivers — a significant bottleneck for large audiences. **Broadcast delivery:** A broadcast packet (destination address such as 255.255.255.255 or a subnet broadcast) is delivered to every host on the local network segment, regardless of whether that host actually wants the data. **Overhead on non-subscribers:** Hosts that never registered interest still incur the interrupt, deserialization, and (until discarded) processing cost of every broadcast frame — a scalability and security concern, which is why broadcast is normally confined to a single LAN segment and blocked by routers. **Use cases:** Unicast replication remains the default when multicast is not supported end-to-end (common on the public Internet); broadcast is largely restricted to local discovery protocols like ARP and DHCP.



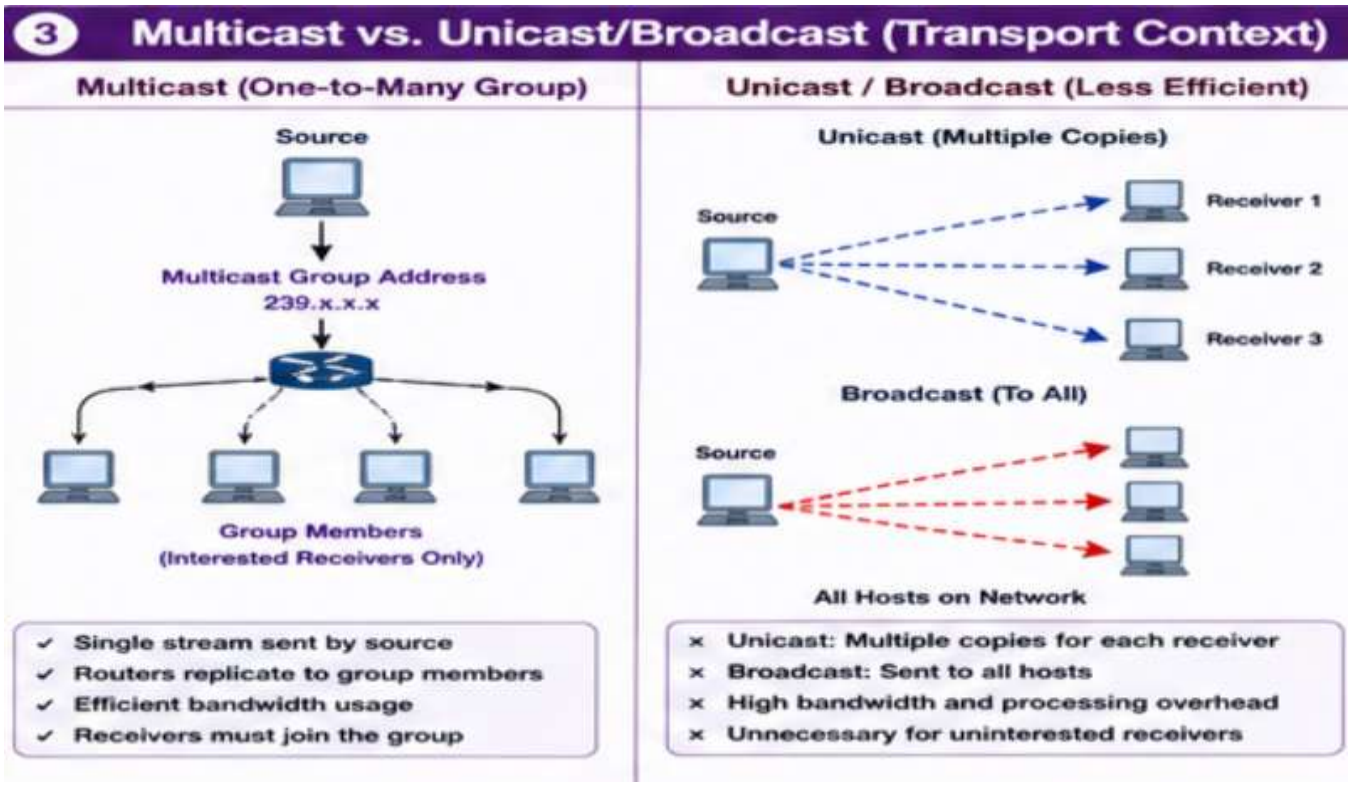


Figure 3. A multicast distribution tree replicating only at branch routers, versus unicast replication (N copies leave the source) and broadcast flooding (every host on the segment receives the packet).

Comparative Synthesis

Multicast is the theoretically optimal solution for one-to-many transport because it pushes the replication cost into the network fabric, where it can be shared, rather than concentrating it at the sender. In practice, multicast requires router and ISP support that is inconsistently deployed across the public Internet, so unicast replication remains the pragmatic fallback for most large-scale streaming services (content-delivery networks effectively approximate multicast's efficiency using edge-caching hierarchies instead). Broadcast, meanwhile, is the least selective of the three and is deliberately scoped to a single local segment precisely because its overhead is imposed on every attached host whether or not it is interested in the data.

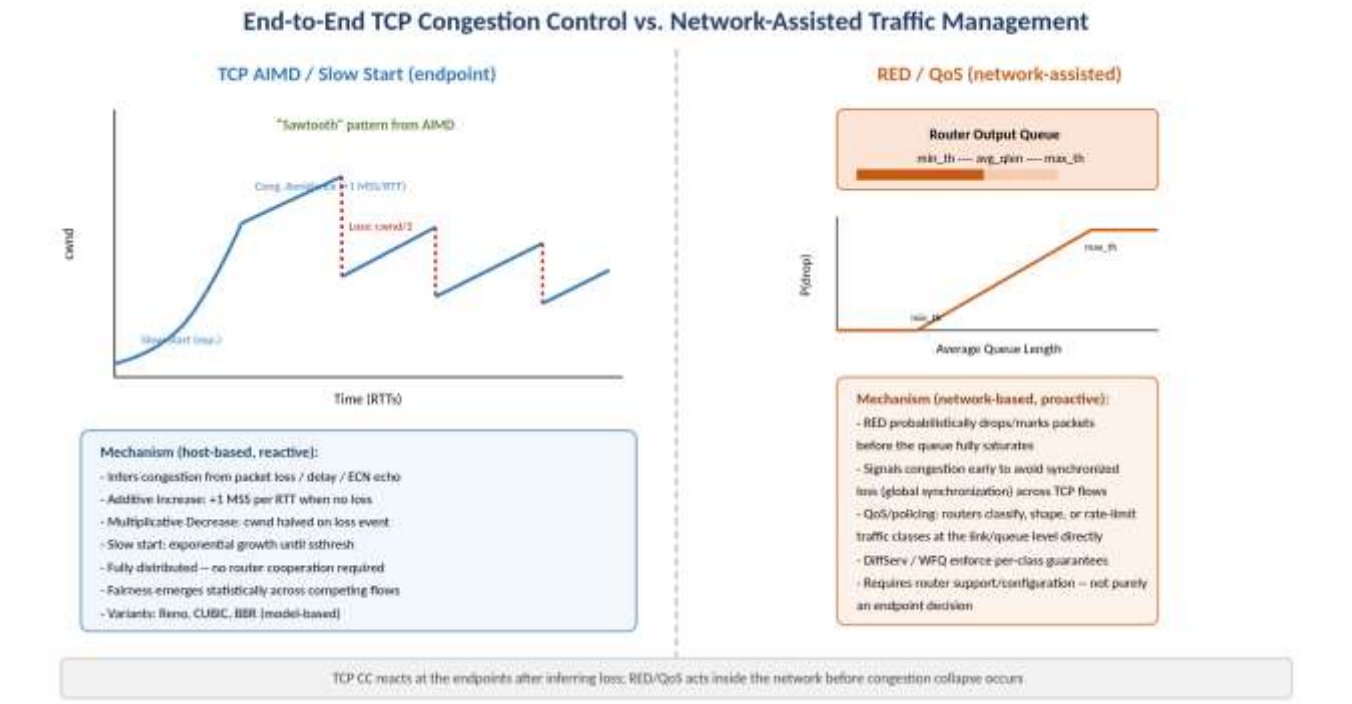
Practical Implications

This is why enterprise and campus networks (where administrators fully control the routing infrastructure) make far heavier use of native IP multicast than the open Internet does, and why Internet-scale 'multicast-like' services are usually built at the application layer instead, through CDN edge caches or dedicated multicast overlay networks running above ordinary unicast.

4. TCP Congestion Control vs. Network Traffic Management

Congestion in a packet network can be managed from two directions simultaneously: the sending endpoints can throttle themselves based on inferred network conditions, or the network's own routers can actively manage their queues to signal or prevent congestion before it becomes severe. TCP relies primarily on the former; RED and QoS mechanisms implement the latter.

Column 1 — End-to-End TCP Congestion Control	Column 2 — Network-Assisted Traffic Management
Slow start: A new TCP connection begins with a small congestion window (cwnd) and doubles it every round-trip time, growing exponentially until either loss occurs or a slow-start threshold (ssthresh) is reached. **Congestion avoidance (AIMD):** Once past slow start, cwnd grows additively — roughly one maximum segment size per RTT — producing linear growth; on detecting loss, cwnd is cut multiplicatively (classically halved), producing the characteristic sawtooth throughput pattern. **Inference from loss/delay:** Because routers rarely tell endpoints directly that they are congested, TCP infers congestion indirectly — from packet loss (timeout or triple duplicate ACK) or, with explicit congestion notification (ECN), from a router-set bit echoed back by the receiver. **Fully distributed:** No coordination between competing flows is required; fairness among similar flows emerges statistically from each independently running the same AIMD logic. **Algorithm variety:** Beyond classic Reno-style AIMD, modern variants such as CUBIC (loss-based, tuned for high-bandwidth paths) and BBR (model-based, estimating bottleneck bandwidth and RTT directly) adjust the same underlying goal through different signals.	**Random Early Detection (RED):** Routers monitor the average queue length and begin probabilistically dropping (or ECN-marking) packets once it exceeds a minimum threshold, with drop probability rising toward a maximum threshold — acting before the queue actually overflows. **Avoiding global synchronization:** By dropping packets from different flows probabilistically and gradually rather than in one burst at tail-drop, RED prevents many TCP senders from all halving their windows in lockstep, which would otherwise cause synchronized throughput oscillation across the whole link. **QoS classification and policing:** Routers can classify traffic into classes (e.g., DiffServ code points) and apply differentiated treatment — priority scheduling, guaranteed minimum bandwidth, or rate-limiting/policing — independent of what any individual TCP sender decides to do. **Active queue management (AQM):** Beyond RED, mechanisms like CoDel and PIE directly target queuing delay rather than queue length, aiming to keep latency bounded regardless of offered load. **Requires network cooperation:** Unlike TCP's purely endpoint-based logic, these mechanisms must be configured and maintained by network operators on the routers themselves — they are not something an end host can unilaterally deploy.



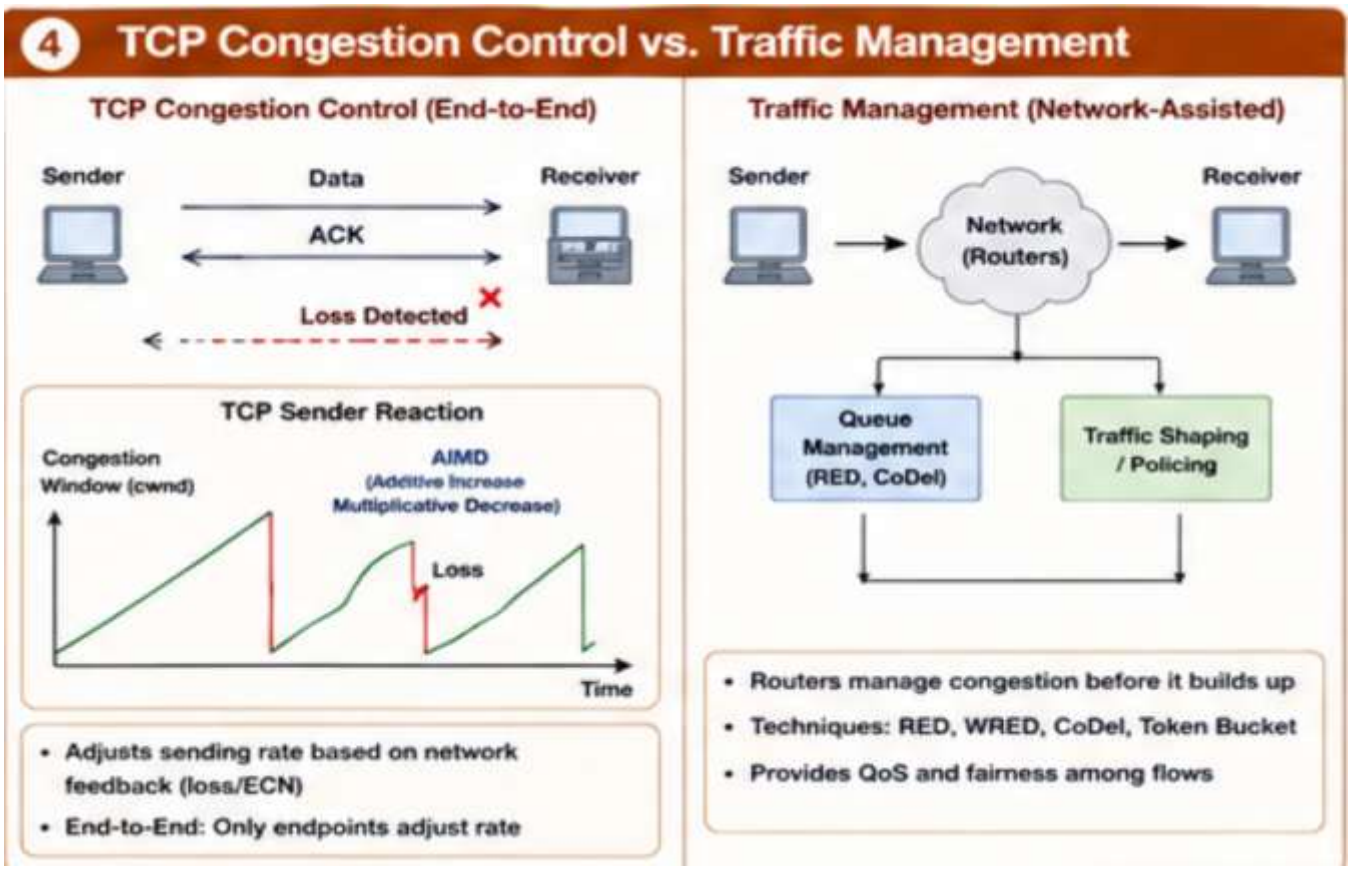


Figure 4. TCP's endpoint-driven AIMD sawtooth (slow start, additive increase, multiplicative decrease on loss) alongside a router's RED drop-probability curve rising between min_th and max_th.

Comparative Synthesis

TCP congestion control and network-assisted traffic management are complementary rather than competing layers of defense against congestion collapse. TCP's endpoint logic is reactive — it responds after congestion has already begun to manifest as loss or delay — and is fully deployable without any router support, which is precisely why it has scaled to the entire Internet. RED, QoS, and AQM mechanisms are proactive, acting inside the network to shape conditions before endpoints even need to react, and they additionally allow operators to enforce policy (bandwidth guarantees, prioritization) that no purely distributed endpoint algorithm could achieve on its own. Well-run networks use both together: endpoint AIMD provides baseline fairness and adaptivity, while router-level AQM keeps queuing delay bounded and prevents pathological synchronized behavior among many competing flows.

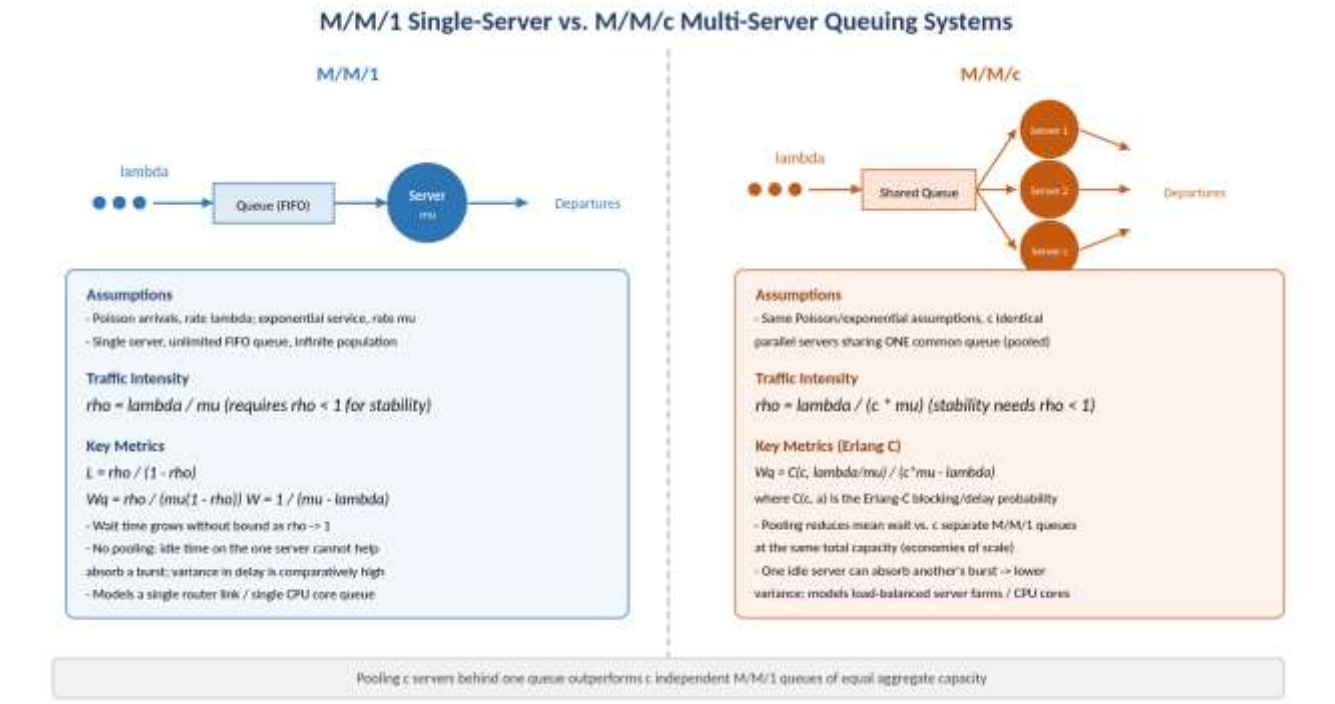
Practical Implications

This layered defense is precisely why bufferbloat became a recognized problem: routers with very large, undifferentiated buffers delayed triggering TCP's own loss-based signal for so long that queuing delay climbed to seconds before AIMD ever reacted — a failure of the network-side half of this partnership that AQM schemes like CoDel were specifically designed to correct.

5. Single-Server (M/M/1) vs. Multi-Server (M/M/c) Queuing Models

Queuing theory provides the analytical language for reasoning about wait times and buffer occupancy at any point where arriving work competes for limited service capacity — a router's output link, a web server's request handler, or a CPU's ready queue. The M/M/1 and M/M/c models capture the single-resource and pooled-resource cases respectively.

Column 1 — M/M/1 Single-Server Model	Column 2 — M/M/c Multi-Server Model
Assumptions: Arrivals follow a Poisson process with rate λ; service times are exponentially distributed with rate μ; there is exactly one server, an unbounded FIFO queue, and an infinite population of potential customers.	**Assumptions:** Same Poisson-arrival, exponential-service assumptions, but with c identical servers operating in parallel, all drawing from one shared, common FIFO queue rather than c separate independent queues.
Traffic intensity: $\rho = \lambda / \mu$. Stability strictly requires $\rho < 1$ — the server must, on average, complete work faster than it arrives, or the queue grows without bound.	**Traffic intensity:** $\rho = \lambda / (c\mu)$. Stability again requires $\rho < 1$, but now the aggregate service capacity is $c\mu$ rather than μ, so the same λ produces proportionally lower utilization per server.
Key metrics: Mean number in system $L = \rho / (1 - \rho)$; mean wait in queue $W_q = \rho / (\mu(1 - \rho))$; mean total time in system $W = 1 / (\mu - \lambda)$ (Little's Law: $L = \lambda W$ ties these together).	**Key metrics (Erlang-C):** The probability an arriving customer must wait is given by the Erlang-C formula $C(c, \lambda/\mu)$; mean queue wait $W_q = C(c, \lambda/\mu) / (c\mu - \lambda)$, noticeably smaller than c independent M/M/1 queues carrying the same aggregate load.
Behavior near saturation: As ρ approaches 1, wait time grows asymptotically without bound — small increases in load near capacity produce disproportionately large delay increases.	**Pooling / economies of scale:** Because all servers draw from one shared queue, an idle server can immediately pick up work that would have queued behind a busy server in a partitioned (non-pooled) design — this is the same principle behind why one shared bank-teller line outperforms c separate single-teller lines.
No pooling benefit: A momentarily idle single server provides no help absorbing a burst of arrivals elsewhere, since there is nowhere else for capacity to come from — variance in individual customer delay is comparatively high.	**Practical relevance:** Directly models load-balanced server farms, multi-core packet processing, and any resource pool where a dispatcher can route work to whichever server is free.



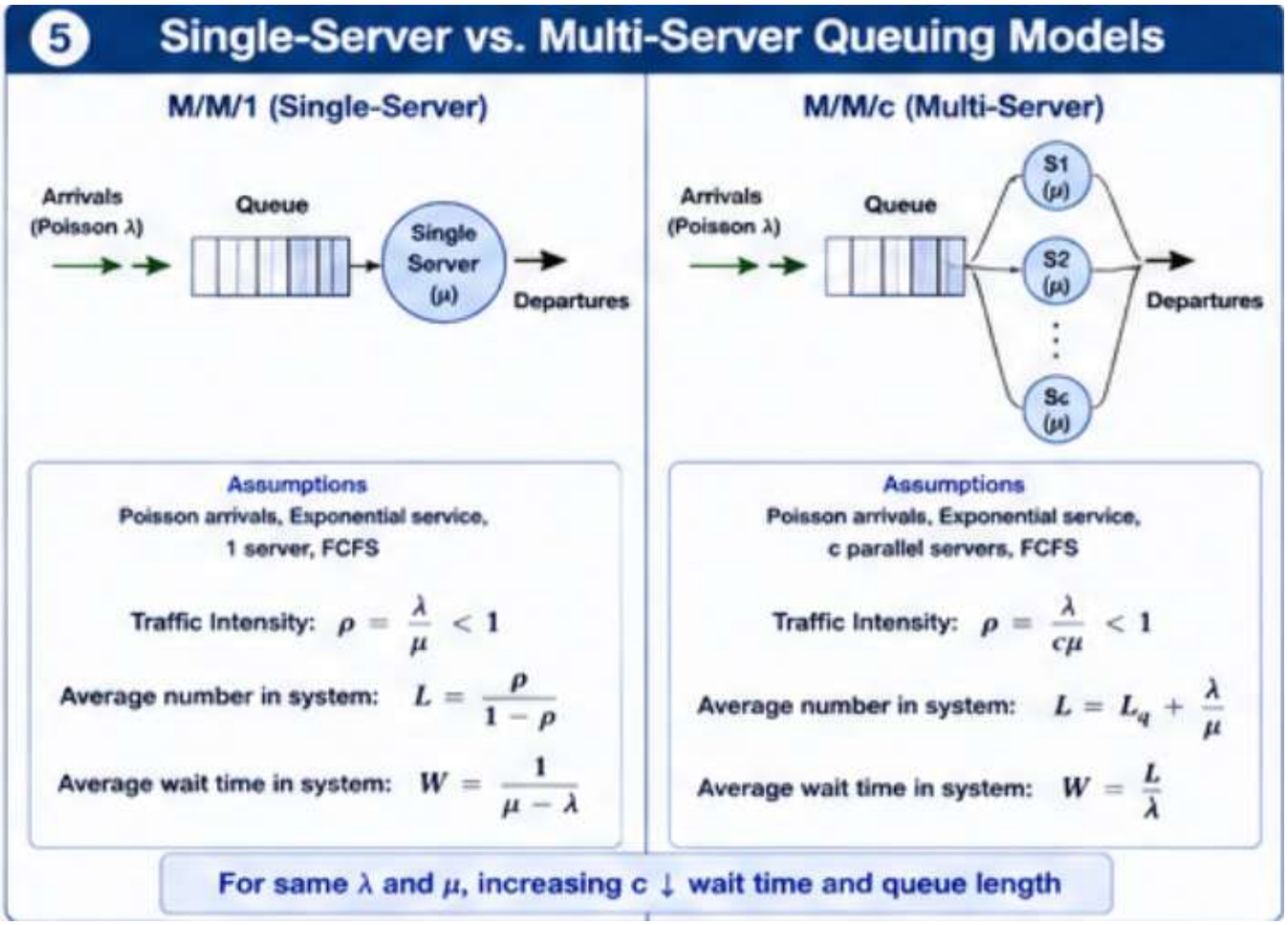


Figure 5. A single arrival stream feeding one FIFO queue and one server (M/M/1) contrasted with the same arrival stream feeding a shared queue serviced by c parallel servers (M/M/c).

Comparative Synthesis

Both models share the same probabilistic foundation — Poisson arrivals and exponential service — but differ in how service capacity is organized, and that organizational difference has real consequences for delay. Pooling c servers behind a single shared queue strictly outperforms operating c independent M/M/1 queues of identical aggregate capacity, because idle capacity on any one server can be applied to relieve congestion anywhere in the pool rather than being stranded behind its own dedicated line. This is precisely why load balancers route requests to a shared worker pool rather than statically partitioning traffic to dedicated per-client servers, and why the M/M/c model, despite its more complex closed-form solution (Erlang-C), better represents most real data-center and networking capacity-planning problems than M/M/1 does.

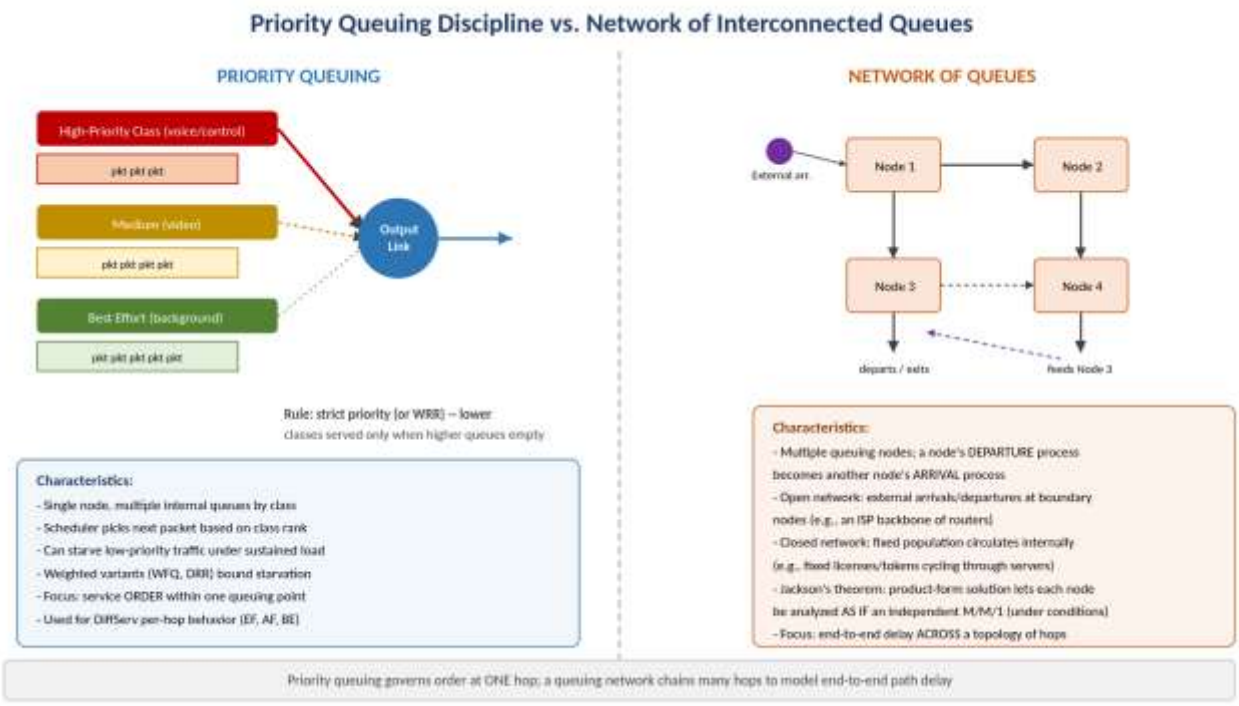
Practical Implications

Capacity planners use this result directly: consolidating several lightly loaded, independently-queued links or servers into one pooled resource of equal total capacity reduces mean wait time without adding any hardware — the gain comes purely from removing the artificial partitioning between queues.

6. Priority Queues vs. Networks of Queues

Beyond the single-queue models above, real systems raise two further questions this section separates cleanly: first, in what order should waiting work be served when not all of it has equal urgency, and second, how does delay accumulate when work must pass through a sequence of queuing points rather than just one.

Column 1 — Priority Queuing Disciplines	Column 2 — Networks of Queues
Class-based service order: Incoming packets are classified (e.g., voice/control traffic, video, best-effort data) and sorted into separate internal queues at a single node; the scheduler chooses which queue to serve next according to a priority ranking rather than strict arrival order. **Strict priority scheduling:** Under strict priority, the highest-priority non-empty queue is always served first — lower-priority traffic is transmitted only when every higher-priority queue is empty at that instant. **Starvation risk:** A persistent, sufficiently heavy stream of high-priority traffic can indefinitely delay lower-priority queues; this is mitigated in practice with weighted variants such as Weighted Fair Queuing (WFQ) or Deficit Round Robin (DRR), which guarantee each class a minimum share of service. **Scope:** Priority queuing governs the internal service order at a single queuing point — it is a local, per-hop scheduling decision, exemplified by DiffServ's per-hop behaviors (Expedited Forwarding, Assured Forwarding, Best Effort). **Design goal:** Minimize latency and jitter for delay-sensitive classes without necessarily changing the total throughput available to the node as a whole.	**Chained queuing nodes:** Traffic frequently passes through several queuing points in sequence — for example, several routers along an end-to-end path — where the departure process of one node becomes the arrival process of the next. **Open vs. closed networks:** An open network has external arrivals entering and departures leaving at its boundary nodes (e.g., traffic entering and exiting an ISP backbone); a closed network has a fixed population that circulates indefinitely among the nodes with no external entry or exit (e.g., a fixed pool of tokens or virtual circuits cycling through processing stages). **Jackson's theorem:** For a broad class of open networks of exponential-service queues with probabilistic routing between nodes, Jackson's theorem shows the network has a product-form solution — under the right conditions each node can be analyzed as if it were an independent M/M/1 queue with an appropriately adjusted arrival rate, dramatically simplifying end-to-end analysis. **Scope:** A network of queues models end-to-end delay across an entire topology of hops, not the service order at any single hop — it answers 'how long does the whole path take,' not 'which packet goes next.' **Combining both models:** In practice, each node in a queuing network may itself use a priority discipline internally, meaning the two concepts operate at different levels of the same system simultaneously.



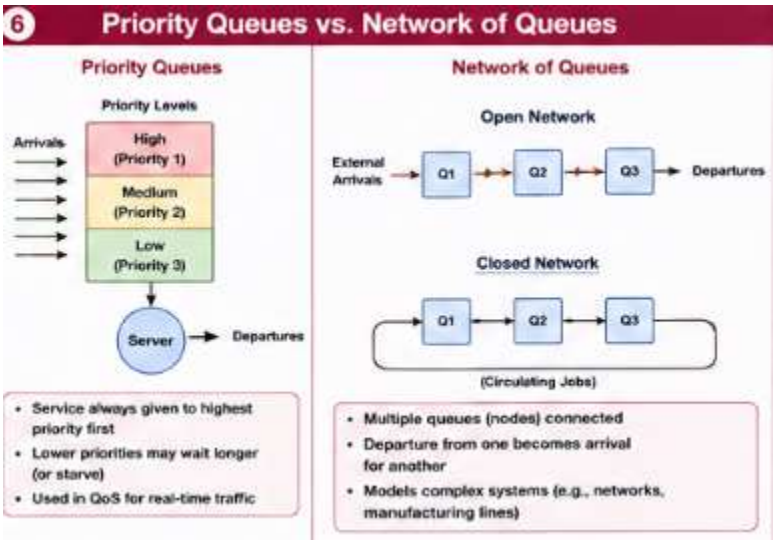


Figure 6. A single node's priority scheduler merging three ranked traffic classes into one output link, versus a topology of interconnected queuing nodes where one node's departures feed another node's arrivals.

Comparative Synthesis

Priority queuing and networks of queues are not competing models but operate at different scopes of the same underlying problem. Priority disciplines answer a local question — given several classes of work waiting at one resource, which is served next — and directly shape per-hop latency and jitter. Networks of queues answer a global question — given a sequence of resources a job must pass through, what is the expected end-to-end delay — and rely on results like Jackson's theorem to make that composition analytically tractable. A realistic model of, say, an ISP backbone typically nests the two: each router in the topology applies its own priority scheduling internally, while the overall path delay experienced by a flow is the composition of delays across every queuing node the network-of-queues framework represents it passing through.

Practical Implications

Network engineers rely on this layering when doing capacity planning end-to-end: per-hop priority configuration is tuned locally at each router to protect latency-sensitive classes, while the overall service-level agreement offered to a customer is validated against the composed, multi-hop queuing-network delay model rather than any single hop in isolation.

7. QUIC Protocol vs. TLS 1.3 Layered on TCP

QUIC and 'classic' TLS-over-TCP both aim to deliver encrypted, reliable, multiplexed application data, but they differ sharply in architecture: QUIC integrates transport, multiplexing, and security into a single protocol running over UDP, while the traditional stack layers TLS 1.3 as an independent module on top of TCP.

Column 1 — QUIC (Integrated, over UDP)	Column 2 — TLS 1.3 over TCP (Layered)
Built on UDP: QUIC runs entirely in user space over UDP, which lets it avoid kernel-level TCP stack ossification and evolve rapidly without waiting on OS updates or middlebox compatibility for TCP option changes. **Integrated handshake:** QUIC combines the transport (connection setup) and cryptographic (TLS 1.3) handshakes into a single exchange, so a new connection typically completes in one round trip, and a resumed connection to a previously visited server can send application data immediately with zero additional round trips (0-RTT). **Independent stream multiplexing:** QUIC natively supports multiple independent byte streams within one connection, each with its own flow control and loss recovery — critically, loss of a packet belonging to one stream does not block delivery of data on other streams. **Eliminating head-of-line (HOL) blocking:** Because QUIC does not impose one single ordered byte stream across all multiplexed data (unlike TCP), a lost or delayed packet affects only the stream it belongs to, directly solving the transport-level HOL blocking problem that persisted even after HTTP/2 introduced application-level multiplexing over TCP. **Connection migration:** QUIC connections are identified by a connection ID rather than the traditional IP/port 4-tuple, allowing a connection to survive a client's network change (e.g., Wi-Fi to cellular) without a full re-handshake.	**Built on TCP:** TLS 1.3 is a distinct protocol layer that assumes and depends on TCP beneath it for reliable, ordered byte delivery — it has no independent transport logic of its own. **Sequential handshakes:** The TCP three-way handshake (SYN, SYN-ACK, ACK) must complete first, consuming one full round trip, before the TLS 1.3 handshake (ClientHello/ServerHello key exchange) can even begin, consuming roughly one more round trip on a full (non-resumed) connection — around two RTTs before application data flows, versus QUIC's one. **Application-level multiplexing only:** HTTP/2 introduced multiple logical streams over one TCP connection at the application layer, but TCP itself still delivers one single ordered byte stream beneath them. **Persistent head-of-line blocking:** Because all multiplexed HTTP/2 streams share that one ordered TCP byte stream, a single lost TCP segment stalls delivery of every stream's data until the retransmission arrives and the stream is back in order — even streams whose own data suffered no loss are blocked waiting. **Mature, widely deployed security:** TLS 1.3 itself is a well-audited, standardized security layer independently reusable by any TCP-based application protocol (HTTPS, SMTP, IMAP), which is a genuine advantage of keeping transport and security as separable, modular layers.

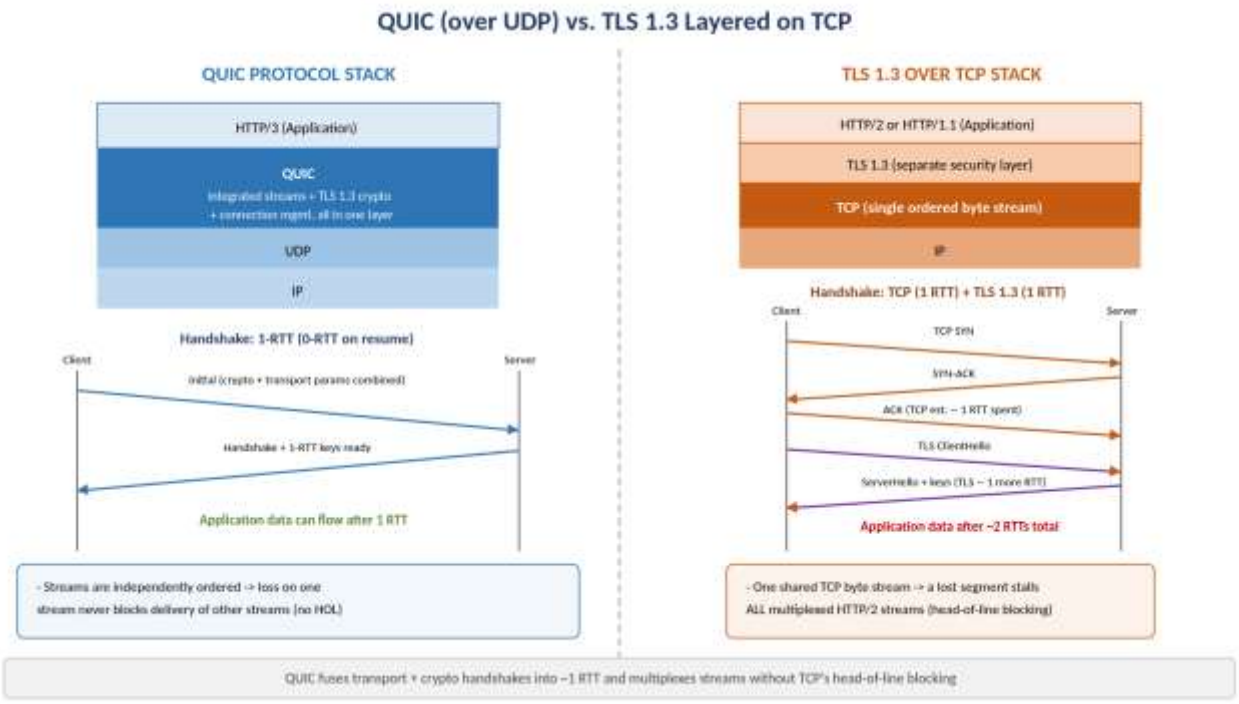
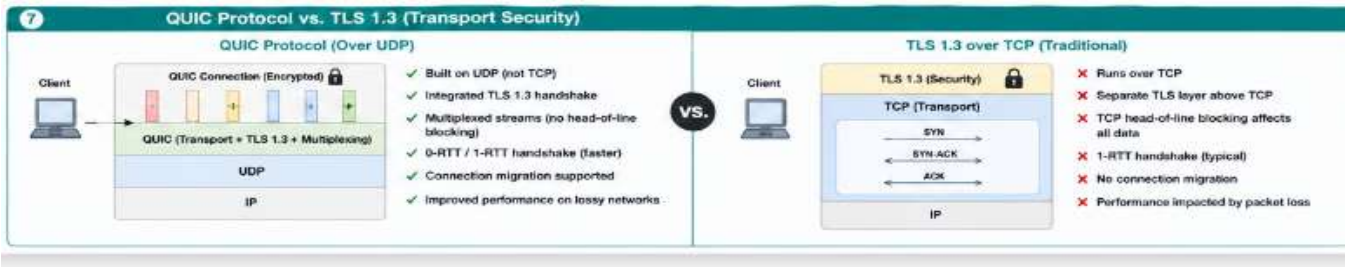


Figure 7. QUIC's fused transport+crypto handshake completing application data delivery in roughly one RTT, versus the sequential TCP handshake followed by a separate TLS 1.3 handshake, totaling roughly two RTTs.



Comparative Synthesis

QUIC's central architectural bet is that fusing transport, multiplexing, and security into one integrated protocol — rather than layering them as independent modules — pays off in both latency and functionality: fewer round trips to first byte, and elimination of transport-level head-of-line blocking that no amount of tuning could remove from TCP plus TLS. The traditional layered approach trades that efficiency for modularity and maturity — TLS 1.3 can secure any TCP-based protocol without modification, and TCP's decades of kernel-level optimization and middlebox familiarity remain available. In practice, both remain in active use: QUIC (as HTTP/3) is now widely deployed for web traffic where its RTT and HOL-blocking advantages matter most, while TLS-over-TCP continues to dominate protocols where UDP is not an option or where QUIC's added complexity is not yet justified.

Practical Implications

This comparison also closes the loop on the report's earlier themes: QUIC's stream multiplexing is a direct descendant of the multiplexing/demultiplexing problem in Section 1, its reliability layer reimplements TCP-like guarantees over UDP as discussed in Section 2, and its loss-recovery logic still relies on AIMD-style congestion

control from Section 4 — illustrating how these transport-layer fundamentals recombine in modern protocol engineering rather than becoming obsolete.